1. Problem Title & Link

Title: LeetCode 525 : Contiguous Array

Link: LeetCode 525 - Contiguous Array

2. Problem Statement (Short Summary)

You are given a binary array:

nums[]

containing only:

0 and 1

Find the maximum length of a contiguous subarray with an equal number of:

0s and 1s

In simple terms:

Find the longest continuous portion where:

count(0)=count(1)

3. Examples (Input → Output)

Example 1

Input:

nums=[0,1]

Output:

2

Explanation:

Subarray:

[0,1]

contains:

1 zero

1 one

Example 2

Input:

nums=[0,1,0]

Output:

2

Explanation:

Possible subarrays:

[0,1]

or

[1,0]
Maximum length:
2

Example 3
Input:
nums=[0,0,1,0,0,0,1,1]
Output:
6

4. Constraints
1 <= nums.length <=10^5
nums[i] is either 0 or 1
Special restrictions:

- Need longest subarray
- Better than $O(n^2)$

5. Core Concept (Pattern / Topic)
Prefix Sum + HashMap

6. Thought Process (Step-by-Step Explanation)
Brute Force
Generate every subarray:

1. Count zeros
2. Count ones
3. Compare counts

Time:
$O(n^2)$
Too slow.

Optimized Thinking
Observation:
Convert:
0 → -1

1 → +1

Now the problem becomes:

Find longest subarray whose sum equals:

0

Why?

Example:

0,1,0,1

↓

-1,+1,-1,+1

Prefix sum:

-1,0,-1,0

When same prefix sum appears again:

Subarray sum =0

meaning:

equal zeros and ones

Algorithm:

1. Replace:

0→-1

2. Maintain prefix sum
3. Store first occurrence in HashMap
4. If prefix repeats:
    - calculate length
    - update answer

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

[0,1,0,1]

Convert:

[-1,+1,-1,+1]

Prefix:

-1 0 -1 0

Repeated:

0 appears twice

Subarray:

[0,1,0,1]
Length:
4

8. Pseudocode (Language Independent)

```
map={0:-1}

prefix=0

maxLength=0

for i from 0 to n-1:

  if nums[i]==0:

    prefix+=-1

  else:

    prefix+=1


  if prefix exists:

    maxLength=
    max(
    maxLength,
    i-map[prefix]
    )

  else:

    map[prefix]=i

return maxLength
```

9. Code Implementation

Python

```
class Solution:

    def findMaxLength(
            self,
            nums):

        prefixMap={0:-1}

        prefix=0

        result=0

        for i in range(
                len(nums)
        ):

            if nums[i]==0:
                prefix-=1
            else:
                prefix+=1

            if prefix in prefixMap:

                result=max(
                    result,
                    i-prefixMap[
                        prefix
                    ]
                )

            else:

                prefixMap[
                    prefix
                ]=i
```

```
    return result
```

Java

```
class Solution {

  public int findMaxLength(
      int[] nums) {

    Map<Integer,Integer>
        map=
        new HashMap<>();

    map.put(0,-1);

    int prefix=0;

    int result=0;

    for(int i=0;
      i<nums.length;
      i++){

      if(nums[i]==0)
        prefix--;

      else
        prefix++;

      if(
          map.containsKey(
              prefix
          )
      ){

        result=
            Math.max(
                result,
```

```
                    i-
                    map.get(
                        prefix
                    )
                );
            }

            else{

              map.put(
                    prefix,
                    i
              );
            }
        }

        return result;
    }
}
```

10. Time & Space Complexity

Time Complexity:

O(n)

Reason:

Array traversed once.

Space Complexity:

O(n)

Reason:

HashMap stores prefix sums.

11. Common Mistakes / Edge Cases

1. Forgetting initial value:

map={0:-1}

Without this:

Subarrays beginning at index 0 fail.

2. Updating duplicate prefix values

Wrong:

map[prefix]=i

Need first occurrence only.

3. Missing:

0→-1

conversion

4. Single element case

[0]

Output:

0

12. Detailed Dry Run (Step-by-Step Table)

Input:

nums=[0,1,0,1]

i	nums[i]	Converted	Prefix	Map	Length
0	0	-1	-1	{-1:0}	0
1	1	1	0	{0:-1}	2
2	0	-1	-1	Found	2
3	1	1	0	Found	4

Result:

4

13. Common Use Cases (Real-Life / Interview)

Balanced signal detection

Equal event analysis

Binary stream processing

Network packet analysis

14. Common Traps (Important!)

1. Forgetting:

0→-1

conversion

2. Updating existing prefix index
3. Missing initial:

0:-1

4. Using nested loops

15. Builds To (Related LeetCode Problems)

Progression:

- LC 525 → Contiguous Array
- LC 560 → Subarray Sum Equals K
- LC 974 → Subarrays Divisible By K
- LC 523 → Continuous Subarray Sum
- LC 930 → Binary Subarrays With Sum

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	O(1)	Too slow
Prefix Array	$O(n^2)$	O(n)	Repeated checks
Prefix Sum + HashMap	O(n)	O(n)	Optimal

17. Why This Solution Works (Short Intuition)

Converting:

0→-1

turns the problem into finding the longest subarray with sum 0. Repeated prefix sums indicate that the values between them cancel out, meaning equal numbers of 0s and 1s.

18. Variations / Follow-Up Questions

1. Return subarray indices
2. Count all valid subarrays
3. Support arbitrary numbers
4. Find shortest balanced subarray
5. Solve for streaming binary data
6.